

equator

User manual

Contents

1. Installation and compiling	2
2. Marking items: <code>check</code> , <code>na</code>	2
2.1. <code>check(id)</code> : a point-marker form	3
3. The checklist grid: <code>render-checklist</code>	6
3.1. Diagnostics	9
3.1.1. Strict mode	12
3.2. <code>excerpt-of</code> : re-emitting the real wording	12
3.3. The page-break idiom	13
4. Style: <code>set-style</code>	14
5. <code>checklist</code> + <code>contexture.bundle</code> : the pilot	17
5.1. Combining with <code>@preview/palimpsest</code>	18
6. Built-in checklists	18

1. Installation and compiling

Import the package, plus `contexture` — the shared backend that assembles the bundle and resolves anchors across documents (see the pilot chapter below; `equator` itself never calls `document(...)`):

```
#import "@preview/equator:0.1.0": *
#import "@preview/contexture:0.1.0": bundle
```

A manuscript plus its completed checklist compiles with one command:

```
typst compile --features bundle --format bundle main.typ
```

This produces `manuscript.pdf` — exactly as it will be submitted, with no visible trace of any `check()/na()` call — and `checklist.pdf`, the completed grid, citing the **real** page numbers `manuscript.pdf` was just laid out with in this same compile. Two optional preview compiles exist alongside it:

```
typst compile --features bundle --format bundle --input preview=true main.typ
```

produces `manuscript-preview.pdf` only (no `checklist.pdf` — see why below), with every `check()`’d span lightly highlighted and tagged with its item id, a drafting aid to see at a glance what’s covered so far and where.

`check()/na()` also work directly in a single ordinary file, with a plain `typst compile`, no bundle involved at all — that form is what every snippet in the next few chapters uses, since it’s the simplest way to show what each function does on its own; the bundle only enters once a real `checklist.pdf` needs to come out of the same compile (see the chapters on the grid and on the pilot).

Why two packages, not one: `contexture` is a small, package-agnostic backend (an anchor primitive, the variant/preview compile axes, diagnostics, and the pilot that turns a list of “satellite documents” into real files) shared with `@preview/palimpsest` (manuscript revision tracking and reviewer response letters) and any future package built the same way. `Equator` builds `checklist(...)` — a description of the completed grid — on top of it; it never calls `document(...)` itself. This is also what lets `equator` and `palimpsest` combine in the very same compile: see `palimpsest`’s manual, “Combining with another `contexture` package”.

2. Marking items: `check`, `na`

`check(id, body)` anchors `body` to item `id` of whichever checklist is active for this compile. Never modifies `body` in the plain compile: zero visual footprint, safe inside any journal template’s flow. Under `--input preview=true`, wraps `body` in a light highlight with the item id superscripted — a drafting aid only, never present in `manuscript.pdf`.

`check(id)` the bare, point-marker form — no `body` at all. Registers item `id`’s location without rendering anything, ever, in the plain compile; `body` must already be displayed by something else. See “`check(id)`: a point-marker form” below for why this exists and when to reach for it instead of `check(id, body)`.

`na(id, reason: none)` declares item `id` not applicable to this manuscript, with an optional justification. Renders nothing wherever it’s called — typically grouped in a dedicated block rather than anchored to any one location in the text (see the examples below).

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

The primary outcome was #check("6a")[change in disease activity score from
baseline to week 12], assessed by a rater blinded to group assignment.
```

Compiled once, plain:

The primary outcome was change in disease activity score from baseline to week 12, assessed by a rater blinded to group assignment.

Once more with `--input preview=true`:

The primary outcome was `change in disease activity score from baseline to week 12`^[6a], assessed by a rater blinded to group assignment.

`check()` doesn't need to know which checklist is active, or even that one exists at all — it just anchors `body` under `id`; whether that `id` means anything is only decided later, when `render-checklist(...)` (or the `checklist(...)` satellite, see below) resolves it against a real checklist's items. This is why `check("6a")` above compiles and renders fine entirely on its own, with no `#show: bundle.with(...)` anywhere in the file. It's also why an `id` that turns out not to belong to **any** item in the active checklist is a diagnostic caught later, in the grid, rather than an error here (see “Diagnostics” below).

An item can legitimately be checked more than once — CONSORT's own guidance for a few items explicitly allows this (e.g., a method described once and its rationale discussed elsewhere) — in which case the grid lists every page it was found on, deduplicated; see “The page-break idiom” below for the one case (a single passage spanning two pages) where deliberately calling `check()` twice on the **same** passage is the recommended way to get a correct, two-page citation.

`na()` is the explicit counterpart: some items genuinely don't apply to a given manuscript (CONSORT's item 8, “Harms” reporting details irrelevant to a trial with no serious adverse events, say). Declaring it explicitly, with a reason, is what turns “item never mentioned” from an oversight worth flagging into a documented, deliberate choice — see the checklist grid's “Not applicable” block further down.

2.1. `check(id)`: a point-marker form

`check(id, body)` renders `body` — that's exactly right the vast majority of the time, since most checklist items simply live at their own spot in the manuscript, unrelated to anything else. Sometimes, though, the location an item covers is already being rendered by something else — most commonly, a revision-tracking package's own marking call, when a reviewer's requested change happens to **be** the manuscript's answer to a checklist item. Calling `check(id, body)` there too, alongside that other call, would render the same text a second time, since it always prints `body` itself. `check(id)`, with no second argument, only ever anchors — exactly like palimpsest's own `passage(anchors, body) / passage(body)` dispatch, one function, two closely related shapes rather than two names to remember.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

#let tiny = (
  name: "TINY",
  full-name: [Tiny reporting checklist],
  items: (
    (section: "Methods", topic: "Outcome assessment", group: none, id: "6a",
      description: [How the primary outcome was assessed.]),
  ),
)

#show: contexture.bundle.with(
  documents: (checklist(checklist: tiny)),
)

// check("6a") with no body registers item "6a" here without rendering
// anything at all --- the sentence itself is printed directly, right
// below, exactly once. Compare with marks-check-basics.typ: same
// manuscript wording, but check(id, body) both anchors AND renders
// (with a highlight box under preview=true); check(id) only anchors.
```

The primary outcome was change in disease activity score from baseline to week 12, assessed by a rater blinded to group assignment. `#check("6a")`

Compiled once, plain — the sentence appears exactly once, exactly as written:

The primary outcome was change in disease activity score from baseline to week 12, assessed by a rater blinded to group assignment.

Once more with `--input preview=true` — a small superscripted id, but no highlight box: there's no body here for a box to wrap around, but the id itself is still visible as a drafting aid:

The primary outcome was change in disease activity score from baseline to week 12, assessed by a rater blinded to group assignment. ^[66]

The item still resolves correctly in `checklist.pdf` — both forms write the exact same "equator-item" metadata shape, so `resolve-item`, `pages-of`, and every diagnostic in `grid.typ` treat a bare `check(id)` identically to `check(id, body)`. The one exception is `excerpt-of`: a bare `check(id)` has no body to quote, so that particular occurrence is skipped when `excerpt-of` gathers text to re-emit (falling back to `on-empty` only if **no** occurrence of `id` has a body at all) — an item covered only via bare markers simply contributes no excerpt, which is the honest answer, not an error. The “blank content” diagnostic never fires on the bare form either: a missing body there is the deliberate point-marker shape, not the mistake it would be for `check(id, body)`’s own empty-body case (`check(id)[]`). See “Combining with `@preview/palimpsest`” below for the real motivating case, and `contexture`’s manual for the general principle behind it.

3. The checklist grid: `render-checklist`

`render-checklist(checklist:, title: auto)` renders the completed grid for `checklist` — every official item, grouped by section (and, when the checklist has them, by a mid-level group inside a section), the resolved page number(s) for each, and a Diagnostics block listing anything that doesn’t add up. It’s exported independently of the pilot (like `palimpsest`’s `change-list()`), for use outside the two-document bundle wiring if ever needed — but the ordinary way to get a `checklist.pdf` is through the `checklist(...)` satellite (see the pilot chapter), which calls this internally.

A checklist is plain data: a dictionary with `name`, `full-name`, and `items` (each with `section`, `topic`, `group` — none when the checklist has no mid-level grouping — `id`, and `description`), plus optional `headers`, `style`, and `citation` fields covered below. `checklist:` has no default anywhere in this package — see the pilot chapter for why an explicit choice is required even though CONSORT ships built in.

The example below is deliberately a tiny, made-up 4-item grid rather than the full 42-item CONSORT checklist, so the whole thing — section band, a mid-level group band, and a `na()`’d item — fits in one page here; a real project passes e.g. `checklists.consort` in `checklist:` instead (see “Built-in checklists” below).

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// A tiny, made-up grid — not a real reporting guideline — kept small so
// checklist.pdf fits on one page for this illustration; a real project
// passes e.g. `checklists.consort` here instead (see the "Built-in
// checklists" chapter).
#let tiny = (
  name: "TINY",
  full-name: [Tiny reporting checklist],
  items: (
    (section: "Title and abstract", topic: "Title", group: none, id: "1a",
      description: [Identification as a randomised trial.]),
    (section: "Methods", topic: "Sequence generation", group: "Randomisation", id: "10",
      description: [Method used to generate the random allocation sequence.]),
    (section: "Methods", topic: "Allocation concealment", group: "Randomisation", id: "11",
      description: [Mechanism used to conceal the allocation sequence.]),
    (section: "Discussion", topic: "Interpretation", group: none, id: "14",
      description: [Interpretation consistent with results, balancing benefits and harms.]),
  ),
)

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (checklist(checklist: tiny).),
)

= Title and abstract

#check("1a")[Effect of Drug X on Outcome Y: a randomised controlled trial.]

= Methods

#check("10")[Participants were randomly assigned via a computer-generated
sequence.]
```

```
#check("11") [Allocation was concealed using sequentially numbered, opaque,  
sealed envelopes.]  
  
= Discussion  
  
This short illustrative manuscript omits a full interpretation section.  
  
#na("14", reason: [Omitted from this compact illustrative example.] )
```

manuscript.pdf, compiled plain:

Title and abstract

Effect of Drug X on Outcome Y: a randomised controlled trial.

Methods

Participants were randomly assigned via a computer-generated sequence.

Allocation was concealed using sequentially numbered, opaque, sealed envelopes.

Discussion

This short illustrative manuscript omits a full interpretation section.

The same manuscript, `--input preview=true` (checklist.pdf is not produced in this compile — see why below):

Title and abstract

Effect of Drug X on Outcome Y: a randomised controlled trial. ^[1a]

Methods

Participants were randomly assigned via a computer-generated sequence. ^[10]

Allocation was concealed using sequentially numbered, opaque, sealed envelopes. ^[11]

Discussion

This short illustrative manuscript omits a full interpretation section.

And checklist.pdf, from the first, plain compile:

Tiny reporting checklist

Section / Topic	No	Description	Reported on page no.
Title and abstract			
Title	1a	Identification as a randomised trial.	1
Methods			
<i>Randomisation</i>			
Sequence generation	10	Method used to generate the random allocation sequence.	1
Allocation concealment	11	Mechanism used to conceal the allocation sequence.	1
Discussion			
Interpretation	14	Interpretation consistent with results, balancing benefits and harms.	N/A

Not applicable

14 — Omitted from this compact illustrative example.

A few things worth noting in that grid: “Randomisation” (item 10/11’s shared group) renders as its own italic band nested inside the “Methods” section band, between the section header and the topic rows — CONSORT 2025’s own layout for its one mid-level heading. Item 14 shows N/A in the Page column, and its justification

appears verbatim in the “Not applicable” block below the table — both come directly from the single `na("14", reason: [...])` call, with no `check()` anywhere for that id.

3.1. Diagnostics

Five distinct situations are flagged, each routed through the same `diagnose(...)` call so `strict: mode` (below) catches all of them the same way:

- an item never `check()`'d or `na()`'d at all — “not covered”;
- a `check()` call whose content is blank (`contexture.is-blank`) — most often a copy-paste mistake where the id was moved but the body wasn't filled in;
- an id used by `check()/na()` that matches no item in the active checklist — typically a typo in the id, or a leftover from switching checklists;
- the same id both `check()`'d and `na()`'d — a real contradiction: an item can't be simultaneously reported somewhere and declared not applicable;
- the same id `na()`'d more than once.

The two per-item cases (blank content, unknown id) and the covered/not-covered/N/A distinction show up right in the Page column, as a  marker; every diagnostic's full sentence — including the two structural ones with no single cell to live in (unknown id, duplicate `na()`) — is listed in full underneath the table, in a dedicated Diagnostics block.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// Isolates every diagnostic case grid.typ can raise, one per line, against
// a tiny made-up checklist so each is trivially visible in checklist.pdf.

#let tiny = (
  name: "TINY-TEST",
  full-name: [Tiny test checklist (not a real reporting guideline)],
  items: (
    (section: "Section A", topic: "Topic 1", group: none, id: "t1",
      description: [Covered normally --- the ordinary, non-diagnostic case.]),
    (section: "Section A", topic: "Topic 1", group: none, id: "t2",
      description: [Never check()'d or na()'d --- demonstrates "not covered".]),
    (section: "Section A", topic: "Topic 2", group: none, id: "t3",
      description: [check()'d with blank content --- demonstrates "blank content".]),
    (section: "Section B", topic: "Topic 3", group: none, id: "t4",
      description: [Both check()'d and na()'d --- demonstrates the conflict diagnostic.]),
    (section: "Section B", topic: "Topic 3", group: none, id: "t5",
      description: [na()'d twice --- demonstrates "declared N times".]),
    (section: "Section B", topic: "Topic 4", group: "Sub-group demo", id: "t6",
      description: [Under a group header, first of a 2-row Topic rowspan.]),
    (section: "Section B", topic: "Topic 4", group: "Sub-group demo", id: "t7",
      description: [Under the same group header, second row of the same Topic rowspan.]),
  ),
)

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 9.5pt)
    body
  },
  documents: (checklist(checklist: tiny),),
)

t1 --- the normal case, no diagnostic expected:
#check("t1") [Normal content, correctly covered.]

t2 --- never covered.

t3 --- blank content:
#check("t3") []

t4 --- both check() and na() on the same id:
#check("t4") [Covered anyway.]
```

```
#na("t4", reason: [Declared not applicable here only to test the conflict
  with check().])

t5 --- na() declared twice for the same id:

#na("t5", reason: [First declaration.])
#na("t5", reason: [Second declaration --- deliberate duplicate.])

t6/t7 --- sharing the "Sub-group demo" group and the same Topic:

#check("t6")[First item of the group demonstration.]
#check("t7")[Second item of the same group, same topic.]

An id entirely unknown to the "tiny" checklist, referenced once via
`check()` and once via `na()`:

#check("zzz")[This id doesn't exist in any grid --- unknown id via check().]
#na("yyy", reason: [Neither does this one --- unknown id via na().])
```

Tiny test checklist (not a real reporting guideline)

Section / Topic	No	Description	Reported on page no.
Section A			
Topic 1	t1	Covered normally — the ordinary, non-diagnostic case.	1
	t2	Never check()’d or na()’d — demonstrates “not covered”.	⚠
Topic 2	t3	check()’d with blank content — demonstrates “blank content”.	1
Section B			
Topic 3	t4	Both check()’d and na()’d — demonstrates the conflict diagnostic.	⚠ N/A
	t5	na()’d twice — demonstrates “declared N times”.	N/A
Sub-group demo			
Topic 4	t6	Under a group header, first of a 2-row Topic rowspan.	1
	t7	Under the same group header, second row of the same Topic rowspan.	1

Diagnostics

⚠ item t2: not covered (no check(), no na())

⚠ item t4: marked not applicable (na) but also checked by check() (p. 1)

⚠ check("t3"): blank content (manuscript p. 1)

⚠ check("zzz"): unknown id for TINY-TEST (manuscript p. 1)

⚠ na("yyy"): unknown id for TINY-TEST (declared p. 1)

⚠ na("t5") declared 2 times

Not applicable

t4 — Declared not applicable here only to test the conflict with check().

t5 — First declaration.

This same example also shows the rowspan behaviour for a run of consecutive items sharing one `topic` (t6/t7, both “Topic 4”, nested under the “Sub-group demo” group): the Topic cell spans both rows rather than repeating.

3.1.1. Strict mode

By default every diagnostic above renders as a soft, visible marker — easy to spot while drafting, but it won't fail a build on its own. `strict: true`, passed to `contexture.bundle(...)` (not to `checklist(...)` — strictness is a property of the whole compile, shared with any other `contexture`-based package in the same bundle, see palimpsest's manual), turns every one of them into a hard compile error instead:

```
#show: contexture.bundle.with(
  strict: true,
  documents: (checklist(checklist: my-checklist)),
)

= Manuscript

// No check("s1") anywhere below --- this manuscript now
// FAILS TO COMPILE under strict: true, with a real compile
// error naming the uncovered item, instead of a soft marker
// sitting quietly in checklist.pdf.
```

The positive case — everything covered, `strict: true` compiles cleanly, with no diagnostic anywhere:

```
#show: contexture.bundle.with(
  strict: true,
  documents: (checklist(checklist: my-checklist)),
)

= Manuscript

#check("s1")[This item is covered here.]
#na("s2", reason: [Not applicable to this manuscript, with a
  proper justification.])
```

A real project typically reserves `strict: true` for a CI compile (a submission-readiness gate), while drafting locally without it, so an incomplete manuscript still produces a readable `checklist.pdf` with markers instead of stopping the build outright.

3.2. excerpt-of: re-emitting the real wording

`excerpt-of(id, quotes: false, show-page: false, on-empty: none)` re-emits the real content `check()` anchored to `id` — one block per occurrence, joined by a paragraph break, so an item checked in two places yields two excerpts. It's not part of the official CONSORT grid, which only ever has a page-number column — this exists for a supplementary appendix some journals or protocols additionally want, with the exact wording quoted next to each item, or for an internal compliance review.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// Demonstrates excerpt-of(): re-emitting the real, verbatim content
// anchored by check(), e.g. for a supplementary "compliance appendix"
// listing exact wording next to each item --- not part of the official
// CONSORT grid (which only has a page-number column), so this lives in
// the manuscript itself, not in checklist.pdf.

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    set text(size: 10.5pt)
    body
  },
  documents: (checklist(checklist: checklists.consort)),
)

= Methods

#check("17a")[The allocation sequence was generated by an independent
  statistician using a computer-based random-number generator.]

#check("18")[Allocation was concealed using sequentially numbered, opaque,
  sealed envelopes.]
```

= Appendix: verbatim compliance excerpts

```
*Item 17a* (Sequence generation): #excerpt-of("17a", quotes: true, show-page: true)

*Item 18* (Allocation concealment mechanism): #excerpt-of("18", quotes: true, show-page: true)

*Item 19* (Implementation) --- never checked in this manuscript, default
`on-empty: none` renders nothing between the colon and the period: #excerpt-of("19").

*Item 20a* (Blinding) --- never checked either, but with an explicit
fallback: #excerpt-of("20a", on-empty: [_ (not reported)]).
```

Methods

The allocation sequence was generated by an independent statistician using a computer-based random-number generator.

Allocation was concealed using sequentially numbered, opaque, sealed envelopes.

Appendix: verbatim compliance excerpts

Item 17a (Sequence generation): **p. 1** — “The allocation sequence was generated by an independent statistician using a computer-based random-number generator.”

Item 18 (Allocation concealment mechanism): **p. 1** — “Allocation was concealed using sequentially numbered, opaque, sealed envelopes.”

Item 19 (Implementation) — never checked in this manuscript, default on-empty: none renders nothing between the colon and the period: .

Item 20a (Blinding) — never checked either, but with an explicit fallback: *(not reported)*.

quotes: true wraps a textual excerpt in real quotation marks, and silently declines on anything `contexture.is-textual` flags as non-text (a figure, a table, a block equation) — the same rule palimpsest’s `pinpoint(quotes: true)` uses. `on-empty` (none by default, i.e. render nothing) is deliberately **not** a diagnostic the way an uncovered item is in the grid: this function can be called from inside the manuscript itself, where no diagnostic may ever render in the real, submitted deliverable — an uncovered item is already flagged exactly once, safely, in `checklist.pdf`.

3.3. The page-break idiom

equator doesn’t try to detect a passage that straddles a page break automatically — there’s no reliable, checklist-agnostic way to tell “one logical passage split across pages” apart from “two genuinely separate mentions of the same item” from inside a structural scan. The recommended idiom instead: call `check()` a second time, with the **same** id, right after the break.

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// The recommended idiom for an item whose content straddles a page break
// (no automatic span detection): a check() call at the start of the
// passage, and a second check() call with the *same* id right after the
// page break. pages-of's dedup means this naturally reports both pages,
// not one merged range and not just the first page.

#show: contexture.bundle.with(
  template: body => {
    set page(paper: "a4", margin: 2.5cm)
    set text(size: 11pt)
    body
  },
  documents: (checklist(checklist: checklists.consort)),
)
```

= Methods

```
#check("13")[
  The intervention consisted of a daily 10 mg dose of Drug X, administered orally with food, for a total of 12 weeks. The comparator was a matching placebo tablet, identical in appearance, taste and packaging, administered on the same schedule.
]

#lorem(200)
#pagebreak()

#check("13")[
  As introduced above, the comparator (placebo) was manufactured by the same facility as Drug X to guarantee identical appearance, and both were dispensed in blister packs labelled only with the participant's allocation number.
]

= Results

#lorem(30)
```

pages-of (used internally by the grid) deduplicates by page, not by call — so this reports both pages, not a merged range and not just the first:

Section / Topic	No	Description	Reported on page no.
Changes to trial protocol	10	Important changes to the trial after it commenced including any outcomes or analyses that were not prespecified, with reason	⚠
Trial setting	11	Settings (e.g., community, hospital) and locations (e.g., countries, sites) where the trial was conducted	⚠
Eligibility criteria	12a	Eligibility criteria for participants	⚠
	12b	If applicable, eligibility criteria for sites and for individuals delivering the interventions (e.g., surgeons, physiotherapists)	⚠
Intervention and comparator	13	Intervention and comparator with sufficient details to allow replication. If relevant, where additional materials describing the intervention and comparator (e.g., intervention manual) can be accessed	1, 2
Outcomes	14	Pre-specified primary and secondary outcomes, including the specific measurement variable (e.g., systolic blood pressure), analysis metric (e.g., change from baseline, final value, time to event), method of aggregation (e.g., median, proportion), and time point for each outcome	⚠
Harms	15	How harms were defined and assessed (e.g., systematically, non-systematically)	⚠
Sample size	16a	How sample size was determined, including all assumptions supporting the sample size calculation	⚠
	16b	Explanation of any interim analyses and stopping guidelines	⚠
Randomisation			
Sequence generation	17a	Who generated the random allocation sequence and the method used	⚠
	17b	Type of randomisation and details of any restriction (e.g., stratification, blocking and block size)	⚠

Two genuinely separate mentions of the same item, landing on the same page, collapse to one page number instead — the same deduplication, the other direction.

4. Style: set-style

Every visual knob `render-checklist` and `check()`'s preview rendering use goes through one shared style mechanism, resolved in three layers, each overriding only what the previous layer left unset:

- 1. equator's own package default (generic, checklist-agnostic — portrait A4, no color, font: auto/text-size: auto meaning "whatever the ambient template already set");
- 2. the active checklist's own `style: field`, if it has one — CONSORT 2025's real landscape A4 layout and column widths, taken from its source `.docx`, is exactly this layer;

3. whatever the compiling author explicitly asked for via `set-style(...)`.

`set-style(...)` takes every knob as a keyword, `auto` by default (meaning “don’t touch this one”); repeated calls merge rather than reset, the same convention as palimpsest’s `set-revisions`:

```
#set-style(
  preview-color: auto,           // check()'s preview-mode highlight color
  show-id: auto,                 // superscript the id in preview mode?
  paper: auto,                   // grid page size, e.g. "a4", "us-letter"
  landscape: auto,               // grid page orientation
  columns: auto,                 // 4 column widths, e.g. (18%, 7%, 1fr, 10%)
  font: auto,                    // grid text font
  text-size: auto,              // grid text size
  header-fill: auto,             // header row background
  header-text-color: auto,
  section-fill: auto,            // section band background
  section-text-color: auto,
  group-fill: auto,              // mid-level group band background
  group-text-color: auto,
)
```

An override on a checklist with no `style`: of its own:

```
#import "../lib.typ": *
#import "../typst-contexture/lib.typ" as contexture

// "tiny" has no style: field of its own, so it renders with equator's
// package-default-style until set-style() below overrides it.
#let tiny = (
  name: "TINY",
  full-name: [Tiny reporting checklist],
  items: (
    (section: "Methods", topic: "Outcome assessment", group: none, id: "1",
      description: [How the primary outcome was assessed.]),
    (section: "Methods", topic: "Randomisation", group: none, id: "2",
      description: [How the allocation sequence was generated.]),
  ),
)

#set-style(
  columns: (22%, 6%, 1fr, 12%),
  header-fill: rgb("#2e3436"),
  header-text-color: white,
  section-fill: rgb("#eeeeec"),
)

#show: contexture.bundle.with(
  template: body => {
    set page(width: 16.6cm, height: auto, margin: 12pt)
    body
  },
  documents: (checklist(checklist: tiny)),
)

#check("1")[The primary outcome was assessed by a rater blinded to group
assignment.]

#check("2")[Participants were randomly assigned via a computer-generated
sequence.]
```

Tiny reporting checklist

Section / Topic	No	Description	Reported on page no.
Methods			
Outcome assessment	1	How the primary outcome was assessed.	1
Randomisation	2	How the allocation sequence was generated.	1

And `preview-color/show-id` also apply to `check()`’s own preview-mode highlight, independently of any checklist (`check()` doesn’t know which checklist is active, by design — see “Marking items” above):

```
#import "../lib.typ": *

#set page(width: 16.6cm, height: auto, margin: 12pt)
#set text(size: 10.5pt)

#set-style(preview-color: rgb("#c01c28"), show-id: false)
```

The primary outcome was `#check("6a")`[change in disease activity score from baseline to week 12], assessed by a rater blinded to group assignment.

The primary outcome was `change in disease activity score from baseline to week 12`, assessed by a rater blinded to group assignment.

These values always win over both equator’s package default and the active checklist’s own `style`: — an explicit ask from the author compiling **this** manuscript is the most specific signal available. A checklist that ships its own faithful `style`: (CONSORT, PRISMA, ...) keeps looking like its real source document by default; `set-style(...)` exists for the cases where that’s not what’s wanted — matching a specific journal’s own house style, or simply personal preference.

5. checklist + contexture.bundle: the pilot

`checklist(checklist:, grid-template: auto)` describes the checklist as a `contexture.satellite(...)` — the value listed under `documents:` in `#show: contexture.bundle.with(...)`. Equator has no pilot of its own: `contexture.bundle(...)` is the single point that ever calls Typst’s `document(...)`, for every package built on it, precisely so stacking this alongside another package’s own satellite (palimpsest’s `letter(...)`, say) never runs into two competing pilots each convinced it alone owns the manuscript/document split.

```
#show: contexture.bundle.with(
  template: my-manuscript-template,
  documents: (
    checklist(checklist: checklists.consort),
    // ... any other contexture-based package's satellite here too
  ),
)

#include "manuscript.typ"
```

`checklist:` has no default, even though CONSORT is the only grid this package ships built in today — forcing an explicit choice costs nothing and avoids a silent, surprising default once a second grid exists (see “Built-in checklists” below). `grid-template: (auto = identity)` is applied to `render-checklist(...)`’s output separately from the manuscript’s own `template:` — most journals want CONSORT’s own official table layout on the checklist page, not the manuscript’s own house style.

Why `checklist.pdf` is never built under `preview: true` or a non-“plain” variant: `check()`’s preview highlighting can shift page breaks (the highlight box adds padding), and so, in principle, could any revision-tracking package’s own tracked-mode marks (`variant() != "plain"`) if one is stacked alongside equator in the same bundle. A grid built from either of those layouts could report page numbers that don’t match the manuscript actually being submitted — worse than not producing one at all. This is `checklist(...)`’s own applicable: `() => contexture.variant() == "plain"` and not `contexture.preview()` rule; `checklist.pdf` only ever comes out of the one, real, plain compile.

Restricting a compile to fewer documents: `--input only=<comma-separated satellite names>` (contexture’s, not equator’s) restricts a single compile to just the manuscript plus the named satellites — `--input only=` with nothing after it produces the manuscript alone, with no checklist at all, useful for a fast preview while drafting a long manuscript where rebuilding the grid every time is unwanted overhead.

Does the checklist compile when the manuscript doesn’t? No — and this is inherent to what a bundle compile is, not a limitation specific to equator: `document(manuscript-name + ..., ...)` and every `satellite’s document(...)` call are all part of the same single Typst compile, so a hard error anywhere in the manuscript

body aborts the whole compile, `checklist.pdf` included. A `check()/na()` **diagnostic** never does this on its own (that’s exactly what `strict: false`, the default, is for) — only a genuine Typst-level error in the manuscript (an undefined function, a malformed table, ...) does.

5.1. Combining with `@preview/palimpsest`

Equator’s `checklist(...)` and `palimpsest`’s `letter(...)` are both just `contexture.satellite(...)` values — listing both under the same `documents:` produces a manuscript, its tracked-changes companion, a reviewer response letter, and a completed CONSORT grid, all from one compile, all citing each other’s real page numbers.

Two rules matter once both packages might touch the same span of text. First: never nest `check(...)` and `passage(...)` inside each other’s body, in either direction — each wraps its own rendering in a `context` block the other’s structural pre-layout scan can’t see through, so nesting either way produces a false diagnostic (see `palimpsest`’s manual, “Combining with another `contexture` package”, for exactly what each failure looks like). Second, and easy to miss because it produces no error or diagnostic at all: don’t call `check(...)` and `passage(...)` as two independent, **rendering** siblings on the exact same wording either — both functions render their own body, so the same text prints twice, plainly, visibly duplicated in `manuscript.pdf`. This only bites when a reviewer’s requested change genuinely **is** the manuscript’s answer to a checklist item; two unrelated spans (the common case) have nothing to duplicate.

When that coincidence happens, use the bare `check(id)` (above) instead of `check(id, body):passage(...)` stays the one call that renders the text and carries its tracked-mode marks, `check(id)` only registers the item’s coverage, with nothing left to duplicate or nest. `contexture`’s own manual walks through this combination end to end, including compiling the tracked-and-preview manuscript together, and states the underlying principle in package-agnostic terms.

6. Built-in checklists

<code>checklists. key</code>	Guideline	Items	Notes
<code>consort</code>	CONSORT 2025 (randomised trials)	42	Landscape A4; one mid-level group, “Randomisation” (17a–21d).
<code>prisma</code>	PRISMA 2020 (systematic reviews)	42	Landscape US Letter; no mid-level groups.
<code>spirit</code>	SPIRIT 2025 (trial protocols)	53	Landscape US Letter.
<code>stard</code>	STARD 2015 (diagnostic accuracy studies)	34	Portrait A4.
<code>strobe.cohort</code>	STROBE (cohort studies)	22	Portrait A4.
<code>strobe.case_control</code>	STROBE (case-control studies)	22	Portrait A4.
<code>strobe.cross_sectional</code>	STROBE (cross-sectional studies)	22	Portrait A4.

Every entry above is transcribed from its official source document (the checklist’s own PDF/Word file), page by page, including its citation and license notice — reproduced verbatim in the “citation” block at the bottom of `checklist.pdf`, and its real column widths and section colors, read directly out of the source `.docx` XML rather than guessed, wherever the source was available as `.docx`. `strobe` is itself a dictionary of the three study-design variants above rather than one single checklist — there is no fourth, “combined” variant covering all three designs in one document, since that source bundles all three designs’ wording into a single item per row, which doesn’t fit the one-description-per-id shape every other checklist here uses.

A checklist is plain data (see “The checklist grid” above) — nothing about `check()`, `na()`, `render-checklist`, or the `checklist(...)` satellite is specific to CONSORT or to any one of the built-in grids above. A project with its own house checklist, or an emerging reporting guideline not yet built into this package, passes its own dictionary of the same shape in `checklist:` instead — exactly the `tiny/TINY` dictionaries used throughout this manual’s own examples.